

Exceptional Control Flow IV

Outline

- Suggested reading Signals
 - Signal Terminology
 - Sending Signals
 - Receiving Signals
 - Blocking and Unblocking Signals
- Suggested reading 8.5

Signals

Signals

- Unix signal
 - A higher-level software form of exception
 - That allows processes and the kernel to interrupt other processes

The kill Program

- Kill program
 - is located in directory `/bin/`
 - sends an arbitrary signal to another process

e.g. `unix> kill -9 15213`

- sends signal 9 (SIGKILL) to process 15213.

Signals Type

- Signal is a message that notifies a process
 - an event of some type has occurred in the system.
- Each type corresponds to some kind of system event
 - Low-level **hardware** exceptions
 - are processed by the kernel's exception handlers would not normally be visible to user processes.
 - Signals provide a mechanism for exposing the occurrence of such exceptions to user processes.
 - Higher-level **software** events
 - in the kernel
 - in other user processes

Hardware Events

- SIGFPE signal (number 8)
 - If a process attempts to divide by zero,
 - then the kernel sends it a SIGFPE signal
- SIGILL signal (number 4)
 - If a process executes an illegal instruction,
 - the kernel sends it a SIGILL signal.
- SIGSEGV signal (number 11)
 - If a process makes an illegal memory reference,
 - the kernel sends it a SIGSEGV signal.

Software Events

- **SIGINT signal (number 2)**
 - While a process is running in the foreground
 - if you type a ctrl-c
 - then the kernel sends a SIGINT signal to the process
- **SIGKILL signal (number 9)**
 - A process can forcibly terminate another process
 - by sending it a SIGKILL signal
- **SIGCHLD signal (number 17)**
 - When a child process terminates or stops,
 - the kernel sends a SIGCHLD signal to the parent.

Linux Signals

> man signal

Number	Name	Default action	Corresponding event
1	SIGHUP	Terminate	Terminal line hangup
2	SIGINT	Terminate	Interrupt from keyboard
3	SIGQUIT	Terminate	Quit from keyboard
4	SIGILL	Terminate	Illegal instruction
5	SIGTRAP	Terminate and dump core (1)	Trace trap
6	SIGABRT	Terminate and dump core (1)	Abort signal from abort function
7	SIGBUS	Terminate	Bus error
8	SIGFPE	Terminate and dump core (1)	Floating point exception
9	SIGKILL	Terminate (2)	Kill program
10	SIGUSR1	Terminate	User-defined signal 1
11	SIGSEGV	Terminate and dump core (1)	Invalid memory reference (seg fault)
12	SIGUSR2	Terminate	User-defined signal 2
13	SIGPIPE	Terminate	Wrote to a pipe with no reader
14	SIGALRM	Terminate	Timer signal from alarm function
15	SIGTERM	Terminate	Software termination signal
16	SIGSTKFLT	Terminate	Stack fault on coprocessor
17	SIGCHLD	Ignore	A child process has stopped or terminated
18	SIGCONT	Ignore	Continue process if stopped
19	SIGSTOP	Stop until next SIGCONT (2)	Stop signal not from terminal
20	SIGTSTP	Stop until next SIGCONT	Stop signal from terminal

Signal Terminology

- Two steps to transfer a signal to a destination process
 - Sending a signal
 - Receiving a signal

Sending a signal

- The kernel
 - **sends** (delivers) a signal to a destination process
 - by updating some state in the context of the destination process.
- The signal is delivered for one of two reasons
 - The kernel has detected a system event
 - Divide by zero, termination of a child process
 - A process has invoked the **kill** function
 - to explicitly request the kernel to send a signal to the destination process.
 - A process can send a signal to itself.

Sending Signals with kill Function

```
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig);
```

returns: 0 if OK, -1 on error

- Will be discussed later

Receiving a signal

- A destination process **receives** a signal
 - when it is forced by the kernel to **react** in some way to the delivery of the signal
- The process can
 - Ignore the signal
 - Terminate
 - Catch the signal
 - by executing a user-level function called a **signal handler**

Pending Signal

- Pending signal
 - A signal that has been sent but not yet received
- Only one
 - At any point in time,
 - there can be at most one pending signal of a particular type
- Not queued
 - If a process has a pending signal of type k
 - then any subsequent signals of type k sent to that process are not queued
 - they are simply discarded.
- A pending signal is received at most once.

Blocking a Signal

- A process can selectively **block** the receipt of certain signals.
- When a signal is blocked
 - it can be delivered
 - but the resulting pending signal will not be received
 - until the process unblocks the signal

Internal Data Structures

- For each process, the kernel maintains
 - the set of pending signals in the **pending bit vector**
 - the set of blocked signals in the **blocked bit vector**
- The kernel sets bit k in **pending**
 - whenever a signal of type k is delivered
- The kernel clears bit k in **pending**
 - whenever a signal of type k is received.

Sending Signals

Process Groups

- To send signals to processes
 - Unix systems provide a number of mechanisms for sending signals to processes
 - All of them rely on the notion of a **process group**
- Every process belongs to exactly one process group
 - which is identified by a positive integer (process group ID).
- By default, a child process belongs to the same process group as its parent.

Process Groups

```
#include <unistd.h>
pid_t getpgrp(void);
```

returns: process group ID of calling process

```
#include <unistd.h>
pid_t setpgid(pid_t pid, pid_t pgid);
```

returns: 0 on success, -1 on error

- Changes the process group of process `pid` to `pgid`
- If `pid` is zero, the PID of the current process is used
- If `pgid` is zero, the PID of the process specified by `pid` is used for the process group ID

setgid Function

e.g. `setpgid(0, 0);`

- Suppose process 15213 is the calling process,
- then this function call
 - creates a new process group whose process group ID is 15213
 - adds process 15213 to this new group.

The kill Program

- Kill program
 - is located in directory `/bin/`
 - sends an arbitrary signal to another process

e.g. `unix> kill -9 15213`

- sends signal 9 (SIGKILL) to process 15213.

The kill Program

- Sending signals to process group by kill
 - Using a negative PID
 - Which causes the signal to be sent to every process in process group PID

e.g. `unix> kill -9 -15213`

- sends a SIGKILL signal to every process in process group 15213.

Sending Signals from the Keyboard

- Job
 - An abstraction used by Unix shells
 - To represent the processes
 - that are created as a result of evaluating a single cmdline
 - At any point, there is at most one foreground job and zero or more background jobs

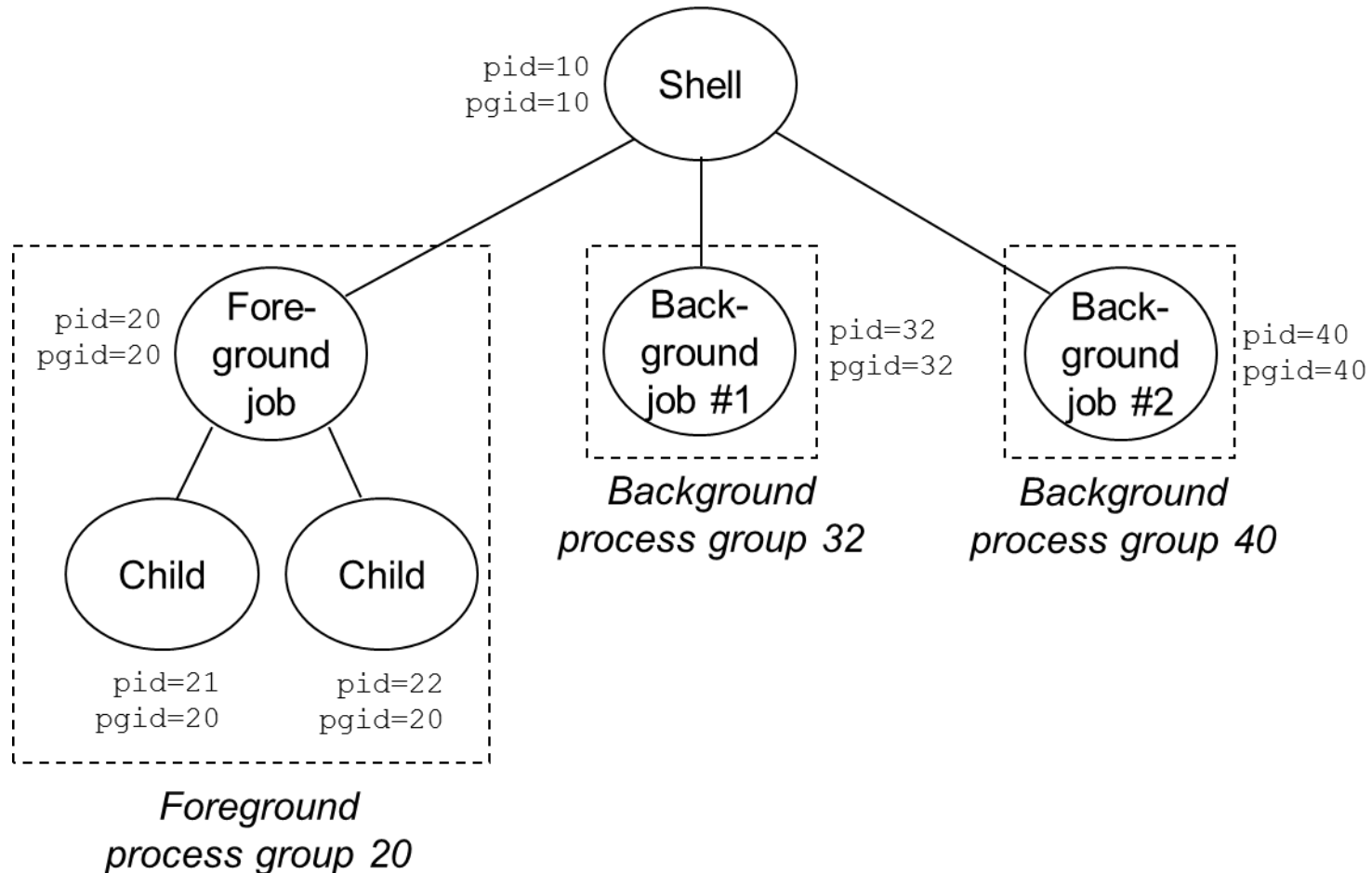
e.g. `unix> ls | sort`

- a foreground job consisting of two processes connected by a pipe

Sending Signals from the Keyboard

- The shell creates a separate process group for each job
 - Typically, the process group ID is taken from one of the parent processes in the job

Sending Signals from the Keyboard



Sending Signals from the Keyboard

- Typing `ctrl-c` at the keyboard
 - causes a `SIGINT` signal to be sent to the shell
 - The shell catches the signal
 - Then sends a `SIGINT` to every process in the foreground process group
 - The result is to terminate the foreground job (default)

Sending Signals from the Keyboard

- Typing `ctrl-z`
 - sends a `SIGTSTP` signal to the shell
 - The shell catches the signal
 - Sends a `SIGTSTP` signal to every process in the foreground process group
 - The result is to stop (suspend) the foreground job (default)

Sending Signals with kill Function

```
#include <sys/types.h>
#include <signal.h>
int kill(pid_t pid, int sig);
```

returns: 0 if OK, -1 on error

- If `pid` is greater than zero,
 - then the `kill` function sends signal number `sig` to process `pid`
- If `pid` is less than zero
 - then `kill` sends signal `sig` to every process in process group `abs(pid)`

```
1  #include "csapp.h"
2
3  int main()
4  {
5      pid_t pid;
6
7      /* child sleeps until SIGKILL signal received */
8      if ((pid = Fork()) == 0) {
9          Pause(); /* wait for a signal to arrive */
10         printf("control should never reach here!\n");
11         exit(0);
12     }
13
14     /* parent sends a SIGKILL signal to a child */
15     Kill(pid, SIGKILL);
16     exit(0);
17 }
```

Sending Signals With the `alarm` Function

```
#include <unistd.h>
```

```
unsigned int alarm(unsigned int secs);
```

returns: remaining secs of previous alarm, or 0 if no previous alarm

- Arranges for the kernel to send a `SIGALRM` signal to the calling process in `secs` seconds.
- If `secs` is zero
 - then no new alarm is scheduled.
- The call to `alarm`
 - cancels any pending alarms
 - returns the number of seconds remaining until any pending alarm was due to be delivered
 - returns 0 if there were no pending alarms.

Receiving Signals

Receiving a Signal

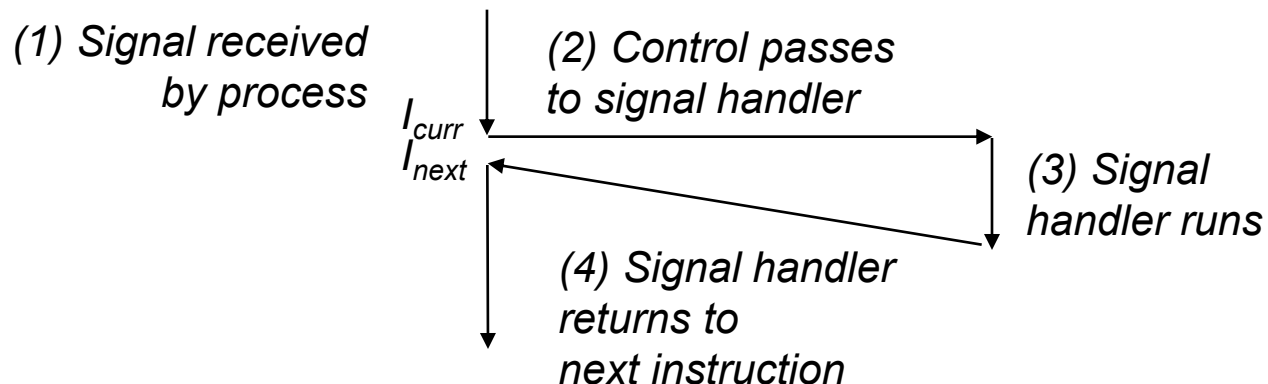
- Preconditions
 - When the kernel is returning from an exception handler
 - and is ready to pass control to process p

Receiving a Signal

- Actions
 - kernel checks the set of unblocked pending signals (**pending & blocked**)
 - If this set is empty (the usual case)
 - then the kernel passes control to the next instruction (I_{next}) in the logical control flow of p.
 - However, if the set is nonempty
 - then the kernel chooses some signal k in the set (typically the smallest k)
 - and forces p to **receive** signal k.

Receiving a Signal

- The receipt of the signal triggers some action by the process.
- Once the process completes the action, then control passes back to the next instruction (I_{next}) in the logical control flow of p.



Receiving a Signal

- Each signal type has a predefined default action, which is one of the following:
 - The process terminates.
 - The process terminates and dumps core.
 - The process stops until restarted by a `SIGCONT` signal.
 - The process ignores the signal.

Receiving a Signal

- `SIGKILL`
 - The default action is to terminate the receiving process
- `SIGCHLD`
 - The default action is to ignore the signal.

Receiving a Signal

- A process can modify the default action associated with a signal
 - by using the **signal** function
 - The only exceptions are SIGSTOP and SIGKILL, whose default actions cannot be changed.

Signal Function

```
#include <signal.h>
typedef void handler_t(int)
handler_t *signal(int signum, handler_t *handler)
    returns: ptr to previous handler if OK,
    SIG_ERR on error (does not set errno)
```

- Three ways to change default actions:
 - If **handler** is **SIG_IGN**, then signals of type **signum** are ignored.
 - If **handler** is **SIG_DFL**, then the action for signals of type **signum** reverts to the default action.
 - Otherwise, change action to **handler** (called signal handler)

Signal Function

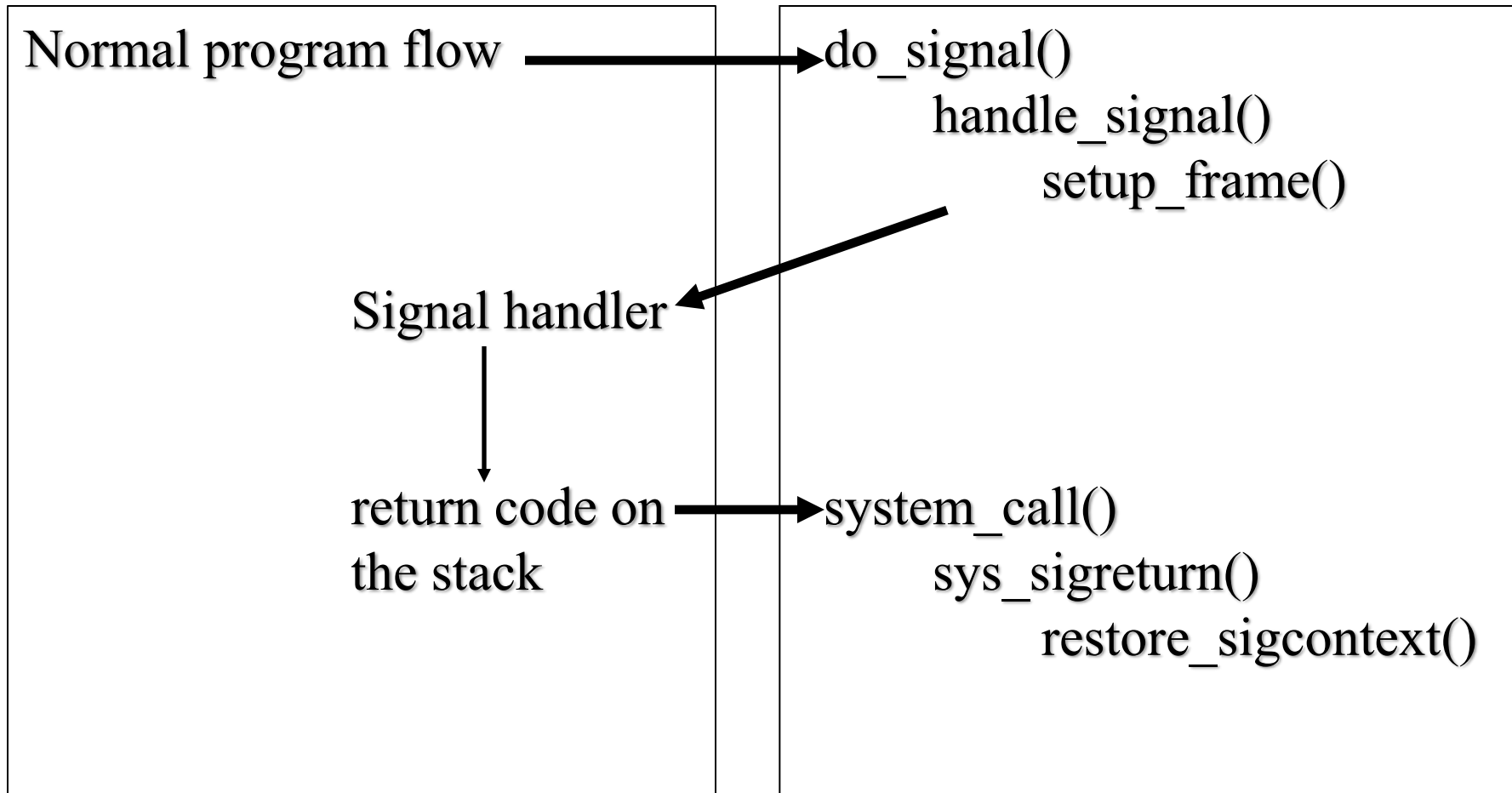
- Installing the handler
 - Changing the default action by passing the address of a handler to the `signal` function
- Catching the signal
 - The invocation of the handler
- Handling the signal
 - The execution of the handler

```
1  #include "csapp.h"
2
3  void handler(int sig) /* SIGINT handler */
4  {
5      printf("Caught SIGINT\n");
6      exit(0);
7  }
8
9  int main()
10 {
11     /* Install the SIGINT handler */
12     if (signal(SIGINT, handler) == SIG_ERR)
13         unix_error("signal error");
14
15     pause(); /* wait for the receipt of a signal */
16
17     exit(0);
18 }
```


Signal Function

- Handling the signal
 - When a process catches a signal of type k ,
 - the handler installed for signal k is invoked
 - with a single integer argument set to k .
 - When the handler executes its **return** statement,
 - control (usually) passes back to the instruction in the control flow
 - where the process was interrupted by the receipt of the signal.
 - in some systems, interrupted system calls return immediately with an error.

Signal Function

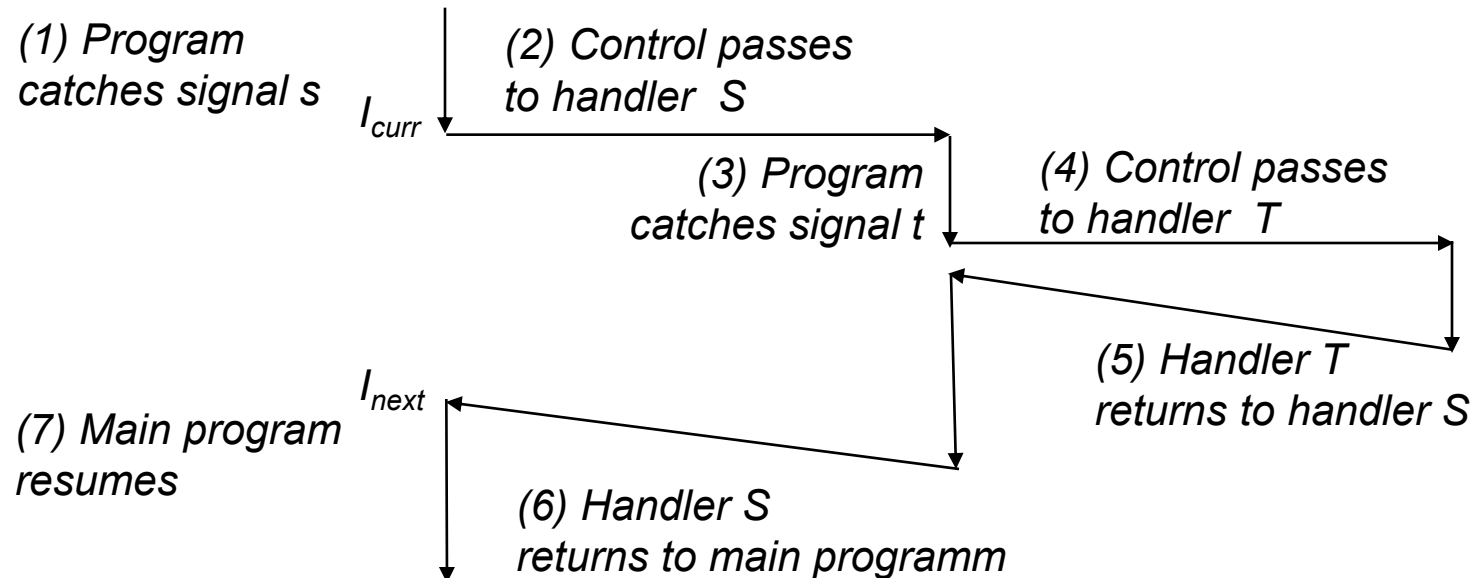


Signal Function

- The same handler function can be used to catch different types of signals
 - By setting different signal numbers as the argument to the signal handler

Blocking and Unblocking Signals

Signal Handlers can be interrupted



$$s \neq t$$

Blocking and Unblocking Signals

- Implicit blocking mechanism
 - Kernel blocks any pending signals of the type currently being processed by a handler
- Explicit blocking mechanism
 - sigprocmask function and its helpers

Blocking and Unblocking Signals

```
#include <signal.h>

int  sigprocmask(int how, const sigset_t *set,
                sigset_t *oldset)
int  sigemptyset(sigset_t *set) ;
int  sigfillset(sigset_t *set);
int  sigaddset(sigset_t *set, int signum);
int  sigdelset(sigset_t *set, int signum);
                                     Return: 0 if OK, -1 on error
Int  sigismember(const sigset_t *set, int signum);
                                     Returns: 1 if member, 0 if not, -1 on error
```

Explicitly Blocking Signals

- `sigprocmask`
 - Changes the set of currently blocked signals
 - `SIG_BLOCK`: add the signals in `set` to blocked ($\text{blocked} = \text{blocked} \mid \text{set}$)
 - `SIG_UNBLOCK`: Remove the signals in `set` from blocked ($\text{blocked} = \text{blocked} \& \sim \text{set}$)
 - `SIG_SETMASK`: $\text{blocked} = \text{set}$
 - If `oldset` is non-NULL, the previous value of the blocked bit vector is stored in `oldset`

Explicitly Blocking Signals

- Signal set
 - Set manipulations are provided by the rest of the functions
 - sigemptyset
 - sigfillset
 - sigaddset
 - sigdelset